The Theory of Computing

Non-Deterministic Finite Automata

A deterministic finite automaton is called deterministic because there must always be exactly one transition leaving a state for every character $\sigma \in \Sigma$.

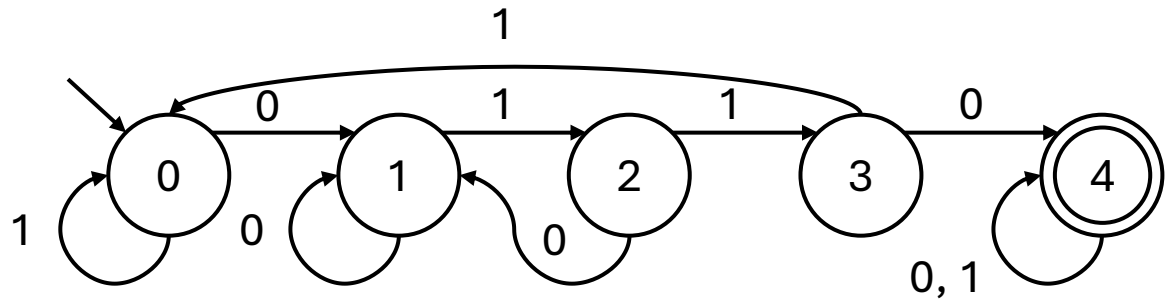
This ensures that there is exactly one computation (sequence of states) for any string w.

In a non-deterministic finite automaton, there can be zero or more transitions leaving any state for each character $\sigma \in \Sigma$.

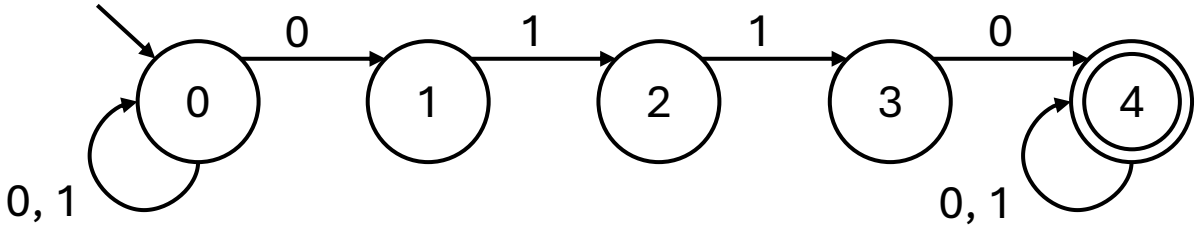
If there are zero transitions leaving a state for the next character, the transition is assumed to lead to a reject state that cannot be escaped.

If there is more than one transition leaving a state for the next character, the input is accepted if any transition leads to an accepting computation.

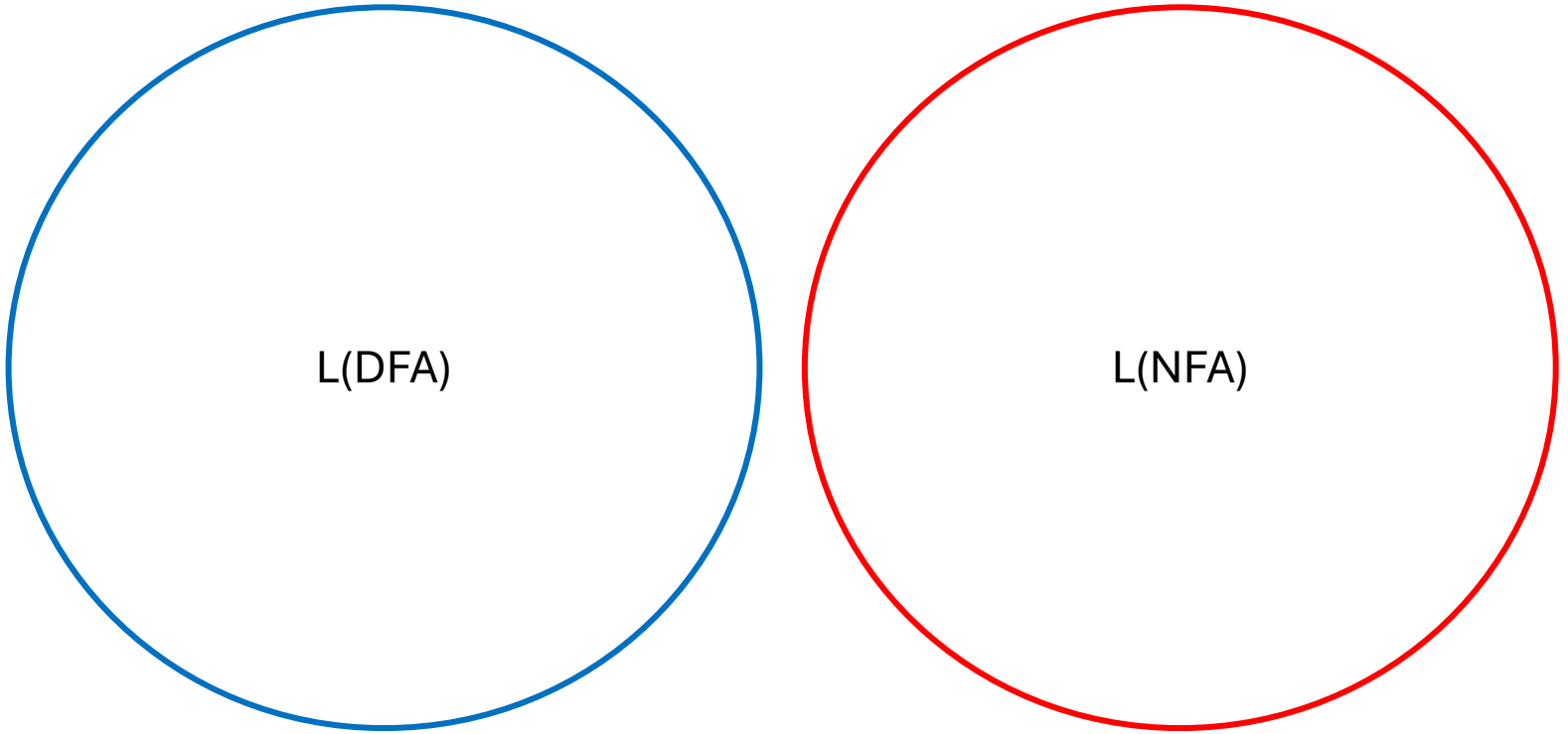
Non-deterministic finite automata can have ϵ-transitions that can be followed without consuming a character of input.



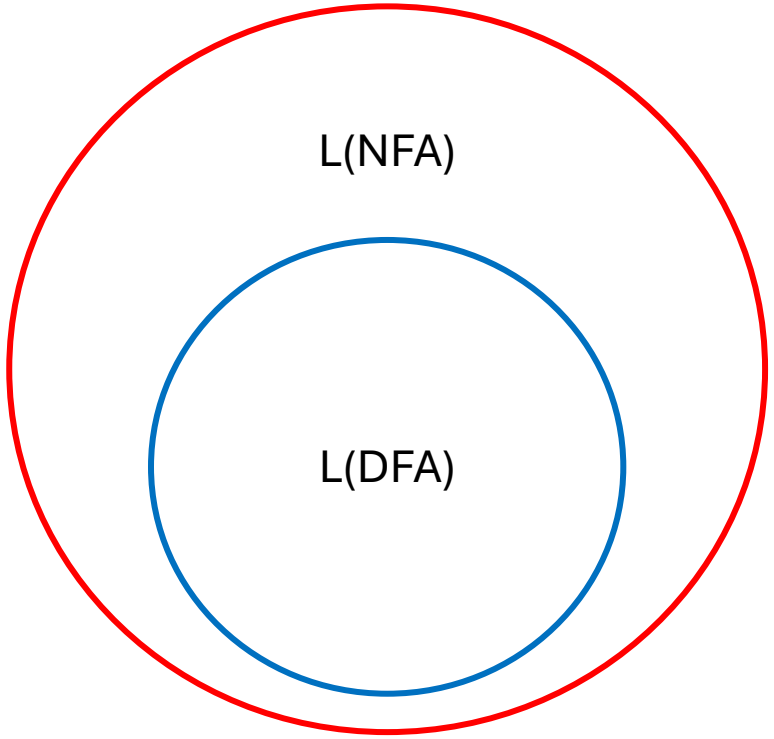
$L = \{w \in \{0,1\}^* \mid w \text{ contains the substring } 0110\}$



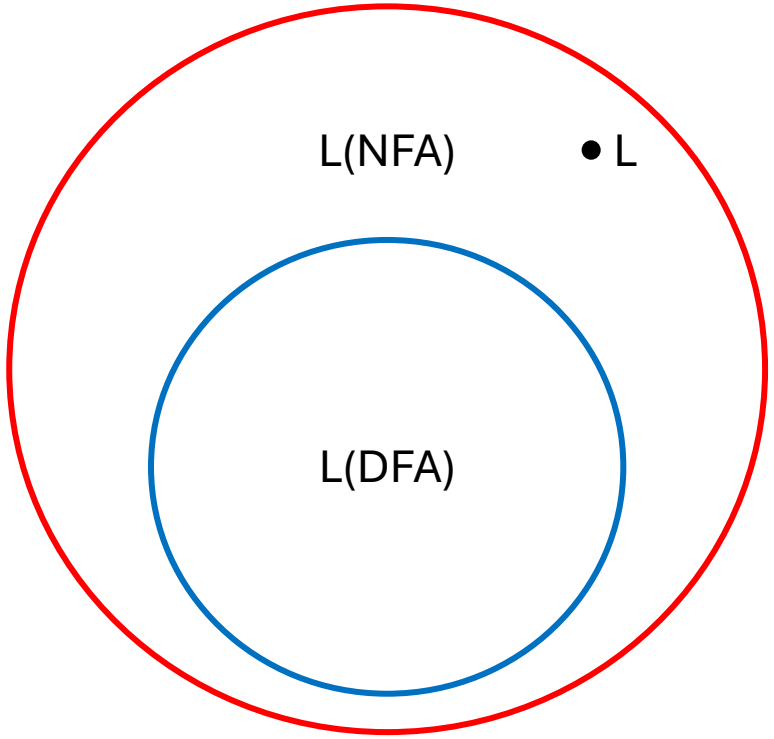
$L = \{w \in \{0,1\}^* \mid w \text{ contains the substring } 0110\}$



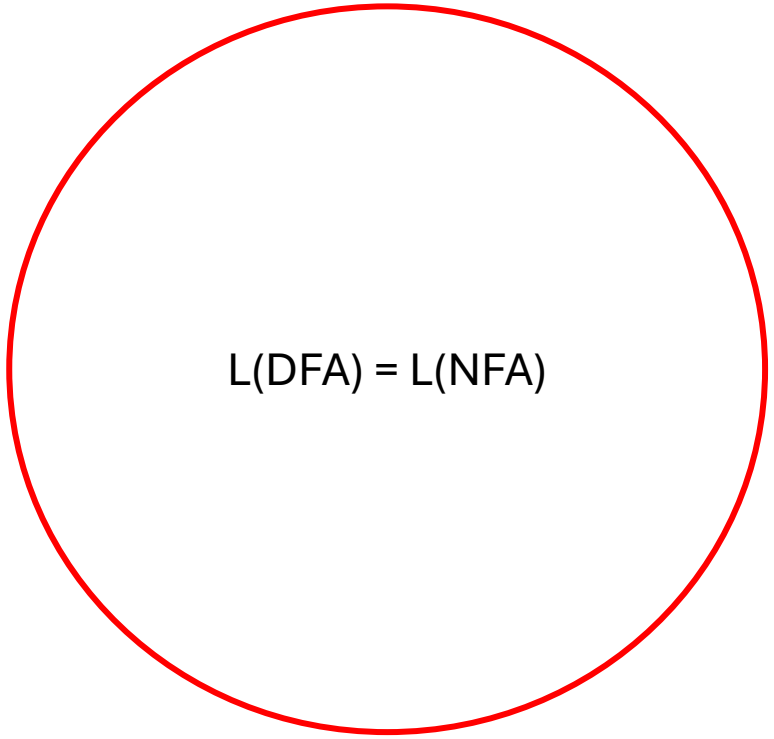
Which model of computation has more power the DFA or the NFA?



Every DFA is also an NFA so DFAs can be no more powerful than NFAs.

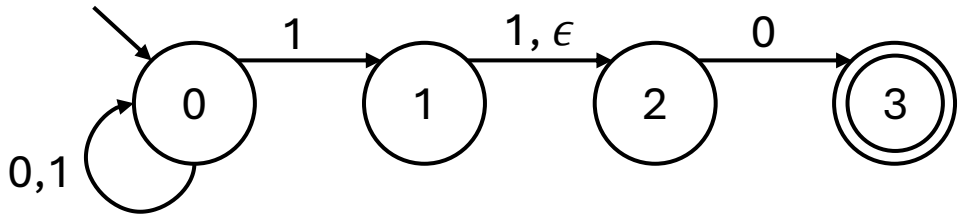


NFAs are more powerful if we can find a language, L, that can be recognized by an NFA but not recognized by a DFA.

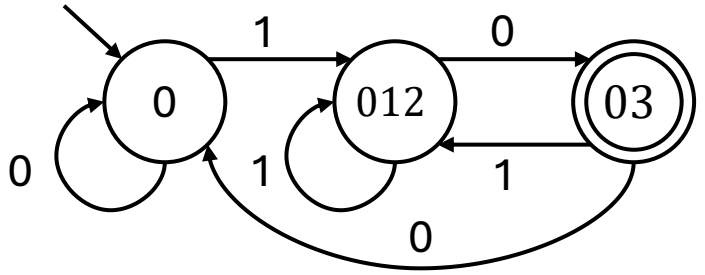


NFAs and DFAs have equal power if we can show that for any NFA, N, we can create a DFA, D, such that $L(N) = L(D)$. We say that D simulates N.

Spoilers: This is the case.



Can we design a DFA that simulates this NFA?



Can we design a DFA that simulates this NFA?

Definition: A non-deterministic finite automaton (NFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:

1. Q is a finite set of *states*,
2. Σ is a finite *alphabet*,
3. $\delta: Q \times \Sigma_\epsilon \to P(Q)$ is the *transition function*[1],
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states* (or *final states*).

[1]Note that $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$.

Theorem: If $L = L(N)$ for some NFA N, then $L = L(D)$ for some DFA D.

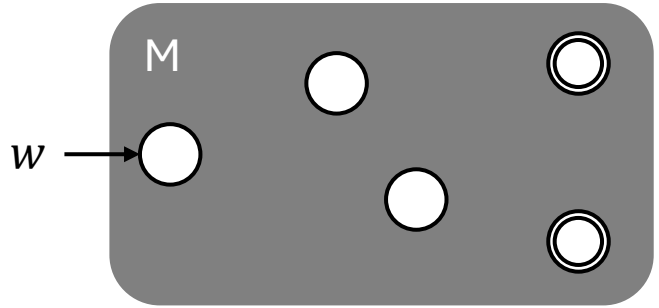
Proof: Let $N = (Q, \Sigma, \delta, q_0, F)$. We will construct $D = (Q', \Sigma, \delta', q_0', F')$ such that $L(D) = L(N)$. Consider defining the following set for a state $R \subseteq Q$.

$$E(R) = \{q \mid q \text{ can be reached from } R \text{ by travelling along 0 or more } \epsilon \text{ arrows}\}$$

1. $Q' = P(Q)$
2. For $R \in Q'$ and $a \in \Sigma$ let

$$\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ for some } r \in R\}$$

3. $q_0' = E(\{q_0\})$
4. $F' = \{R \in Q' \mid R \text{ contains some } q \in F\}$ □

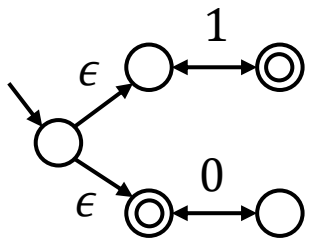


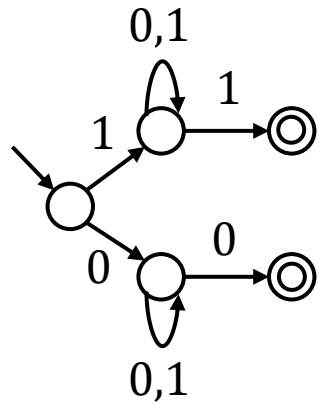
Definition: A language, L, is called a regular language, if $L = L(M)$, for some deterministic finite automaton M.

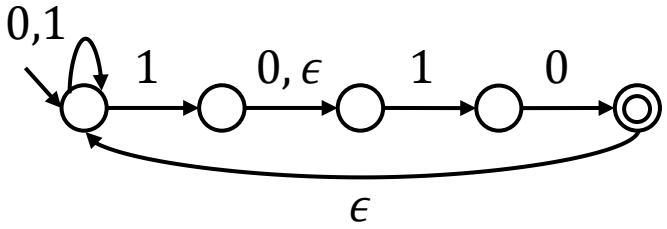
Definition: A language, L, is called a regular language, if $L = L(M)$, for some non-deterministic finite automaton M.

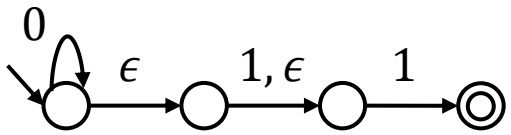
The Theory of Computing

Converting NFAs to DFAs









Can we design DFAs that simulate these NFAs?

The Theory of Computing

Closure of the Regular Languages

Definition: Let L_1 and L_2 be languages. We define the regular operations on languages as follows:

Union: $L_1 \cup L_2 = \{x \mid x \in L_1 \text{ or } x \in L_2\}$

Concatenation: $L_1 \cdot L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$

Star: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$

Consider these languages over the alphabet $\Sigma = \{0,1\}$:

$$L_1 = \{x \mid x \text{ has at least two } 1s\}$$
$$L_2 = \{x \mid |x| \text{ is even}\}$$

Then we can define the following languages:

$$L_1 \cup L_2 = \{x \mid x \text{ has at least two 1s or } |x| \text{ is even}\}$$
$$L_1 \cdot L_2 = \{xy \mid x \text{ has at least two 1s and } |y| \text{ is even}\}$$
$$L_1^* = \{x \mid x \text{ has at least two 1s or } x = \epsilon\}$$
$$L_2^* = \{x \mid |x| \text{ is even}\}$$

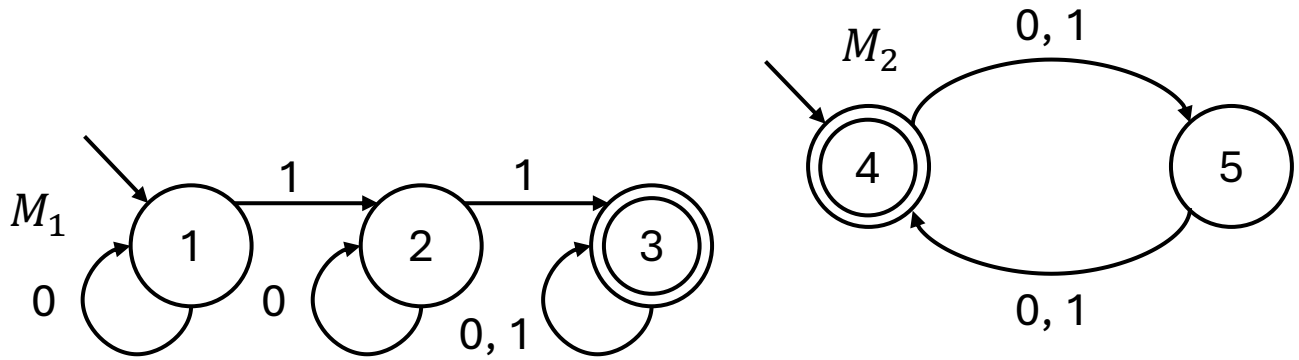
Definition: Given a set S and an operation $\cdot$, we say that S is closed under $\cdot$, if and only if for all $s_1, s_2 \in S$, $s_1 \cdot s_2 \in S$.

Example: The set of even numbers is closed under addition.

Example: The set of odd numbers is not closed under addition.

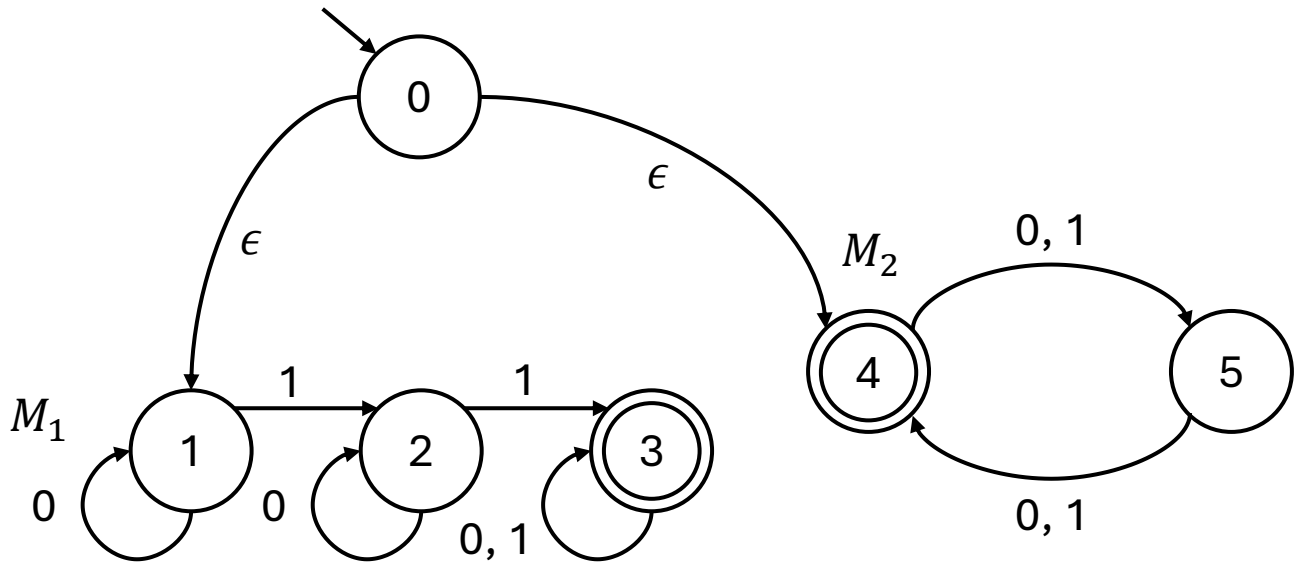
Theorem: The set of regular languages is closed under union ($\cup$), concatenation ($\cdot$) and Kleene star($*$).

Lemma 1: If L_1 and L_2 are regular languages then $L_1 \cup L_2$ is a regular language.



$$L_1 = \{x \mid x \text{ has at least two } 1s\}$$

$$L_2 = \{x \mid |x| \text{ is even}\}$$



$$L_1 \cup L_2 = \{x \mid x \text{ has at least two 1s or } |x| \text{ is even}\}$$

Lemma 1: If L_1 and L_2 are regular languages then $L_1 \cup L_2$ is a regular language.

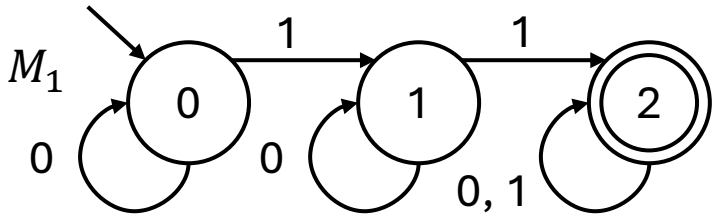
Proof: Assume M_1 is a DFA that recognizes L_1 where $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$.
Assume M_2 is a DFA that recognizes L_2 where $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$.

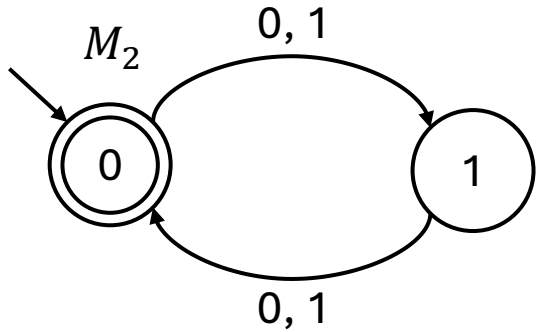
We will construct an NFA, $M = (Q, \Sigma, \delta, q_0, F)$, that recognizes $L_1 \cup L_2$.

1. $Q = Q_1 \cup Q_2 \cup \{q_0 | \, q_0 \notin Q_1 \text{ and } q_0 \notin Q_2\}$
2. Σ is assumed to be the same in M, M_1 and M_2.
3. δ is created by combining δ_1 and δ_2 and adding the transitions $\delta(q_0, \epsilon) = \{q_1, q_2\}$.
4. q_0 is the start state.
5. $F = F_1 \cup F_2$

$L(M) = L_1 \cup L_2$. (Proof of correctness omitted.) □

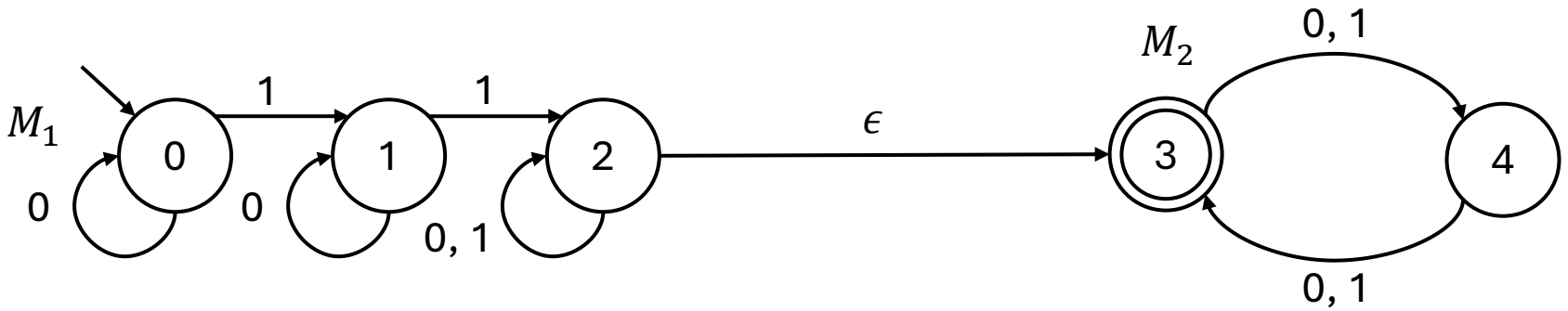
Lemma 2: If L_1 and L_2 are regular languages then $L_1 \cdot L_2$ is a regular language.





$$L_1 = \{x \mid x \text{ has at least two } 1s\}$$

$$L_2 = \{x \mid |x| \text{ is even}\}$$



$$L_1 \cdot L_2 = \{xy \mid x \text{ has at least two 1s and } |y| \text{ is even}\}$$

Lemma 1: If L_1 and L_2 are regular languages then $L_1 \cdot L_2$ is a regular language.

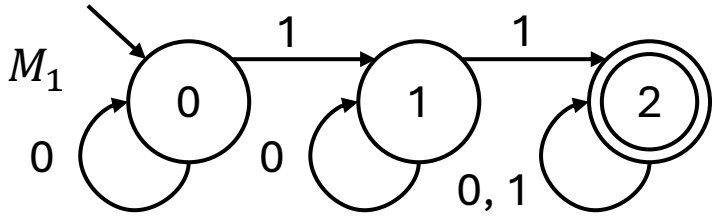
Proof: Assume M_1 is a DFA that recognizes L_1 where $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$.
Assume M_2 is a DFA that recognizes L_2 where $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$.

We will construct an NFA, $M = (Q, \Sigma, \delta, q_1, F)$, that recognizes $L_1 \cdot L_2$.

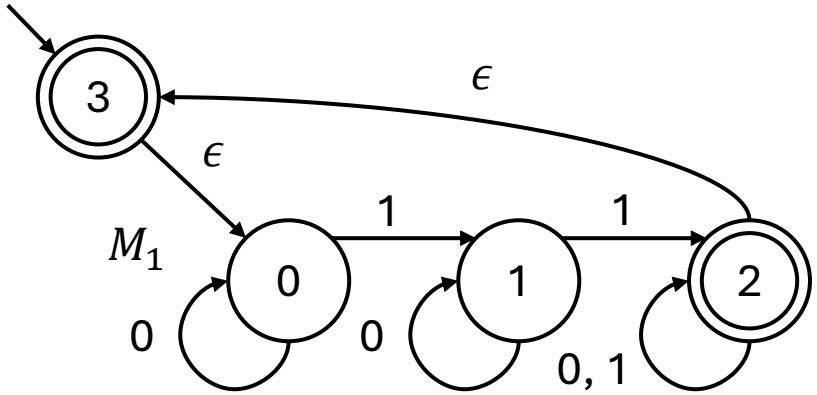
1. $Q = Q_1 \cup Q_2$
2. Σ is assumed to be the same in M, M_1 and M_2.
3. δ is created by combining δ_1 and δ_2 and adding the transitions $q_2 \in \delta(f, \epsilon)$ for all final states $f \in F_1$.
4. q_1 is the start state.
5. $F = F_2$

$L(M) = L_1 \cdot L_2$. (Proof of correctness omitted.) □

Lemma 3: If L_1 is a regular language then L_1^* is a regular language.



$$L_1 = \{x \mid x \text{ has at least two } 1s\}$$



$$L_1^* = \{w_1 \cdots w_k \mid w_i \text{ has at least two } 1s \text{ and } k \geq 0\}$$

Lemma 1: If L is a regular language then L^* is a regular language.

Proof: Assume M_1 is a DFA that recognizes L where $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$.

We will construct an NFA, $M = (Q, \Sigma, \delta, q_0, F)$, that recognizes L^*.

1. $Q = Q_1 \cup \{q_0 | \, q_0 \notin Q_1\}$
2. Σ is assumed to be the same in M and M_1.
3. δ is created by starting with δ_1 and adding the transitions $q_0 \in \delta(f, \epsilon)$ for all final states $f \in F_1$ and $\delta(q_0, \epsilon) = q_1$.
4. q_0 is the start state.
5. $F = F_1 \cup \{q_0\}$

$L(M) = L_1^*$. (Proof of correctness omitted.) □

Definition: Let L_1 and L_2 be languages. We define additional (not regular) operations on languages as follows:

Complement: $\overline{L_1} = \{w \mid w \notin L_1\}$

Intersection: $L_1 \cap L_2 = \{w \mid w \in L_1 \text{ and } w \in L_2\}$

Reversal: $L_1^R = \{w_n \cdots w_2 w_1 \mid w_1 w_2 \cdots w_n \in L_1\}$

Exercise: Prove the regular languages are closed under:

Complement: Swap accepting and rejecting states.

Intersection: Keep track of all states occupied at once.

Reversal: Start in final states and end in the start state.

The Theory of Computing

Regular Expressions

Definition: R is a regular expression if R is:

1. $\sigma \in \Sigma$
2. ϵ
3. $\emptyset$
4. $(R_1 \cup R_2)$ where R_1 and R_2 are regular expressions.
5. $(R_1 \cdot R_2)$ where R_1 and R_2 are regular expressions.
6. (R_1^*) where R_1 is a regular expression.

$$L(00 \cup 1001) = \{00,1001\}$$

$$L(0^*) = \{w \mid w \text{ has 0 or more 0s}\}$$

$$L\big((0110)^*\big) = \{w \mid w \text{ is 0 or more copies of } 0110\}$$

$$L\left((00 \cup 1001)^*\right) = \{w \mid w \text{ is 0 or more copies of 00 or 1001}\}$$

For convenience we use shorthand expressions:

1. If $\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_n\}$ is the alphabet then we use Σ as a stand in for $(\sigma_1 \cup \sigma_2 \cup \cdots \cup \sigma_n)$.
2. R^+ is a stand in for $R \cdot R^*$ meaning one or more copies of R.
3. R^k is a stand in for a concatenation of k Rs with one another.

$$L\left((\Sigma^7)^*\right) = \{w \mid |w| \text{ is divisible by } 7\}$$

Exercises

$L_0 = \{w \in \{0,1\}^* \mid w \text{ contains a single } 1\}$

$L_1 = \{w \in \{0,1\}^* \mid w \text{ has at least one } 1\}$

$L_2 = \{w \in \{0,1\}^* \mid w \text{ contains the substring } 101\}$

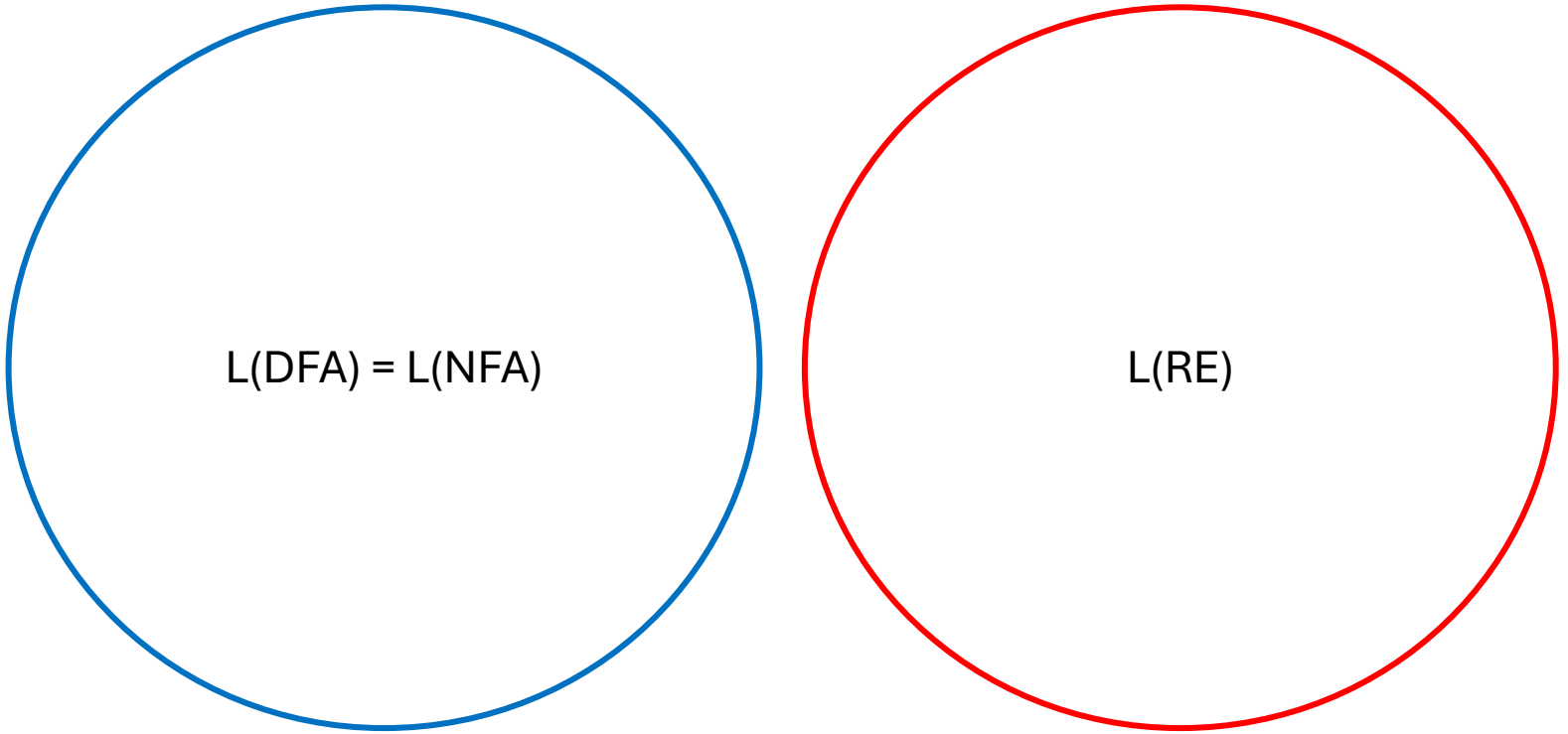
$L_3 = \{w \in \{0,1\}^* \mid w \text{ has even length}\}$

$L_4 = \{w \in \{0,1\}^* \mid \text{the length of } w \text{ is divisible by } 3\}$

$L_5 = \{w \in \{0,1\}^* \mid w \text{ starts and ends with the same character}\}$

The Theory of Computing

Regular Expressions and Regular Languages



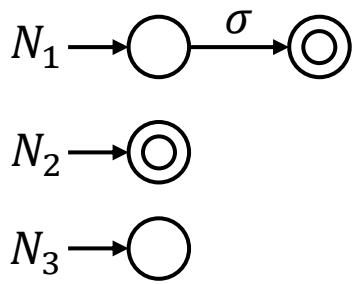
Which model of computation has more power the DFA/NFA or the RE?

Theorem: If R is a regular expression then there is an NFA N such that

$$L(R) = L(N)$$

Proof: We will consider the six types of regular expression given in the definition and construct an NFA N for each of them.

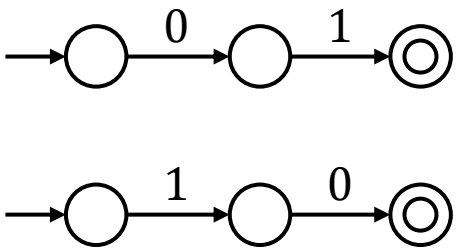
1. $R = \sigma$ for some $\sigma \in \Sigma$.
2. $R = \epsilon$
3. $R = \emptyset$
4. $R = R_1 \cup R_2$
5. $R = R_1 \cdot R_2$
6. $R = R_1^*$



For the final three cases we will build NFA for R_1 and R_2 and then apply the appropriate closure construction. □

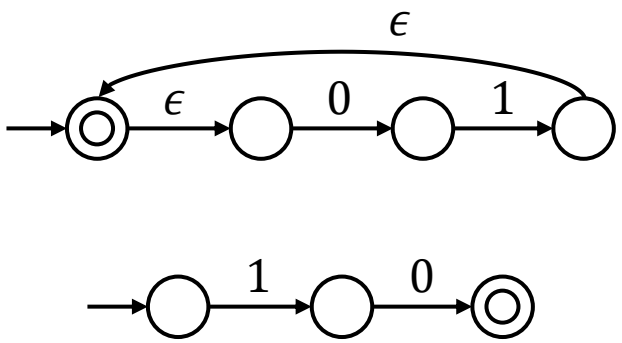
Convert this RE to an NFA using the construction just given.

$$(01)^* \cup 10$$



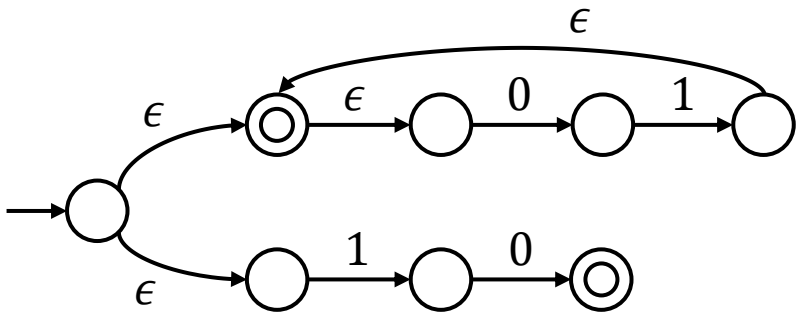
Convert this RE to an NFA using the construction just given.

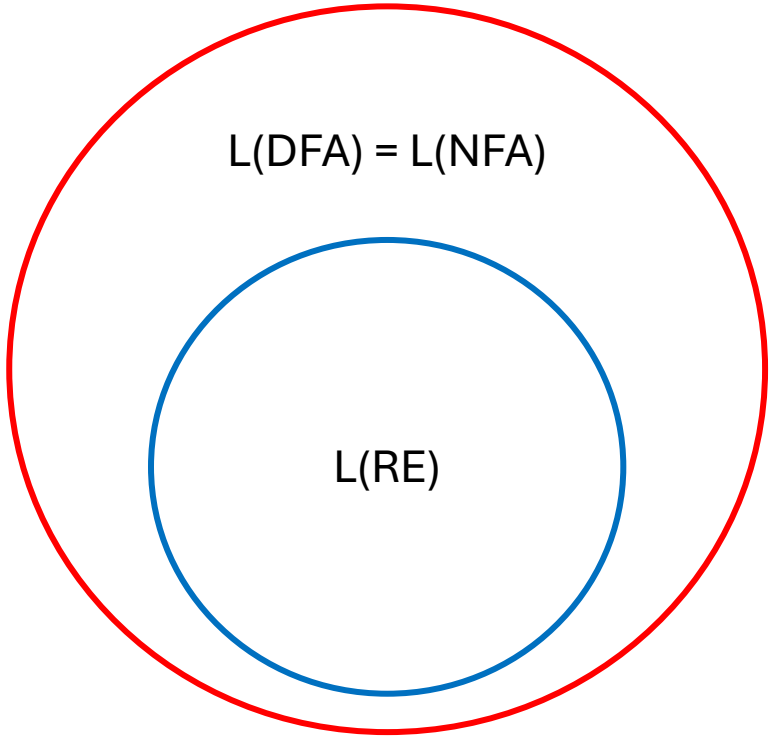
$(01)^* \cup 10$



Convert this RE to an NFA using the construction just given.

$$(01)^* \cup 10$$





Every RE can be converted into an NFA so REs can be no more powerful than NFAs (or DFAs).

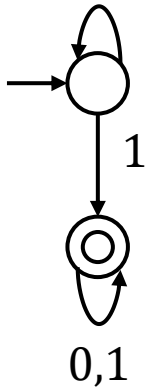
Theorem: If N is an NFA then there is a regular expression R such that

$$L(N) = L(R)$$

Proof: We will convert an NFA into an RE.

We will use an NFA where transitions can be labeled with regular expressions as an intermediate, called a generalized deterministic finite automata (GNFA).

Let’s convert this NFA as an example to demonstrate our construction.



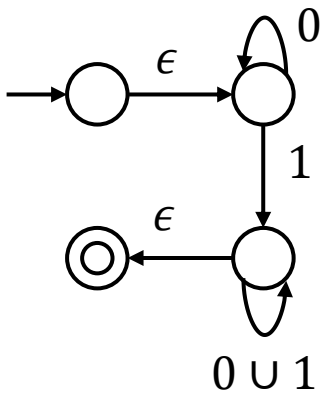
Theorem: If N is an NFA then there is a regular expression R such that

$$L(N) = L(R)$$

Proof (continued): The first step of our conversion will be to take an arbitrary NFA and convert it into a GNFA with these characteristics:

1. The start state has no incoming transitions.
2. There is only a single accept state and it has no outgoing transitions.
3. We add all other missing transitions labeled with a $\emptyset$.

Let’s convert this NFA as an example to demonstrate our construction.

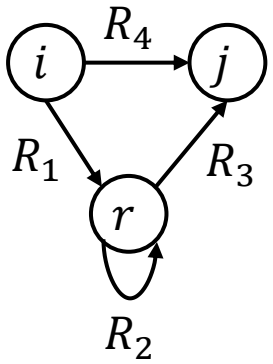


Theorem: If N is an NFA then there is a regular expression R such that

$$L(N) = L(R)$$

Proof (continued): We will convert the GNFA to a RE by removing a single state at time until we only have two states the start state and the accept state. The RE labeling the transition between these states will be R.

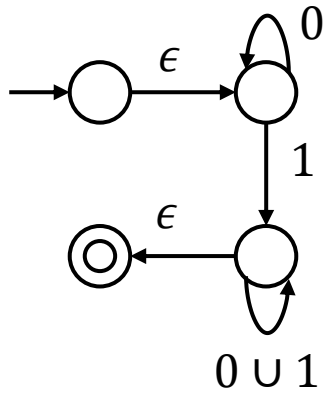
Consider three states in a GNFA in this configuration:

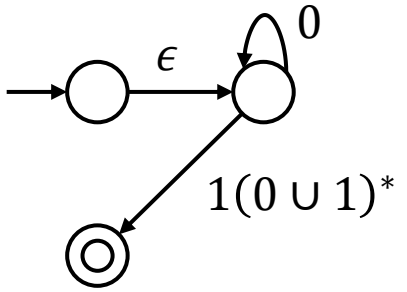


After removing state r the transition from i to j will be relabeled with:

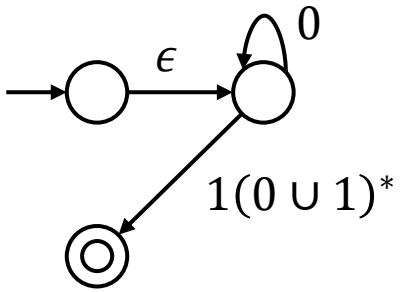
$$R_1 R_2^* R_3 \cup R_4$$

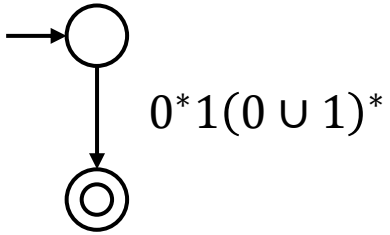
Let’s convert this NFA as an example to demonstrate our construction.

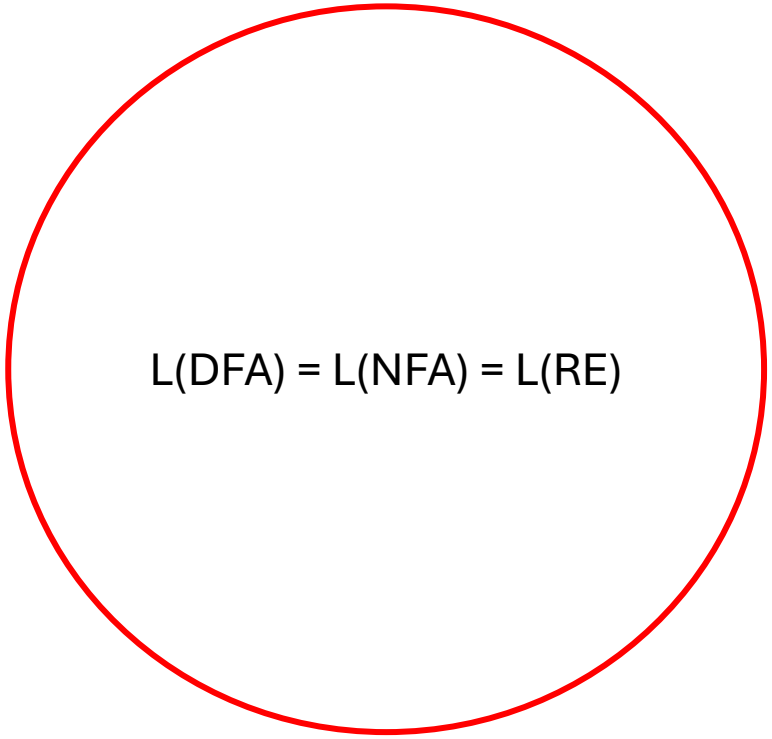




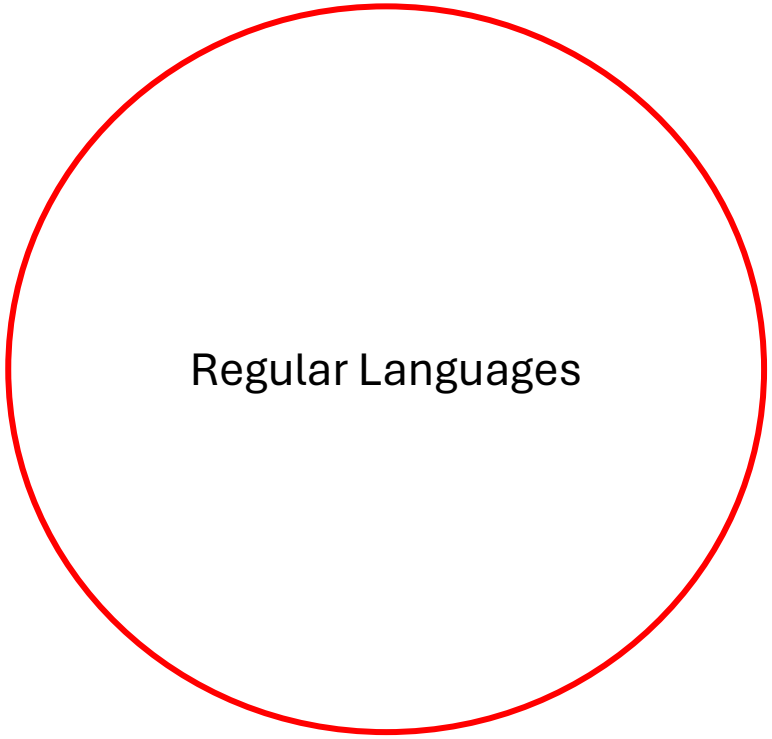
Let’s convert this NFA as an example to demonstrate our construction.







NFAs, DFAs and REs have equal power.



We call these languages the regular languages.

Exercises

Convert this NFA to a RE:

